SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE: CSA1304**

COURSE OUTCOME COVERED

CO2: De**ign Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)**

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:20
Duration : 1 Hour

AT3 - Scenario Based

Code of Conduct:

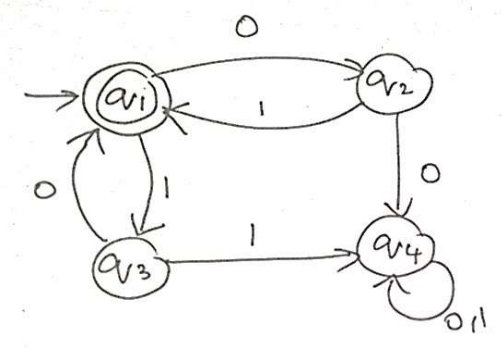
I GIFTSON KEVIN MATTHEW.Y(192511167) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIFTSON KEVIN MATTHEW.Y

SIMATS ENGINEERING
ASSESSMENT TOOL

1. Given a DFA used for validating a specific PIN pattern bbbc, bca ,c, ca, caaaaaa minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

2. Demonstrate how a given finite automaton can be converted into a regular expression



3. Consider the following grammar

S->aSc|T|U|e

T-> bTc|e

U->aUb|e

A} What is the language defined by the grammar?

B)Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

4. I)Show that if L1 and L2 are regular then (L1+l2) also regular

Ii)Is the language { a^p| p is prime} regular? Justify

SIMATS ENGINEERING
ASSESSMENT TOOL

:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized Answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING
ASSESSMENT TOOL

Question 1. Minimize a DFA for PIN Pattern Validation

Answer:

A Deterministic Finite Automaton (DFA) is used to check whether an input string satisfies a particular pattern. DFA minimization reduces the number of states without changing the language accepted by the automaton.

The question lists the PIN patterns bbbc, bca, c, ca, and caaaaaa.

Steps involved in minimization

1. Identify the initial state and the final states of the given DFA.
2. Divide the states into two groups: final states and non-final states.
3. Compare the transitions of states within each group for every input symbol.
4. Separate states whose transitions lead to distinguishable groups.
5. Merge equivalent states and construct the minimized DFA.
6. Verify that the minimized automaton accepts the same valid PIN patterns as the original DFA.

How state reduction improves embedded systems

- Memory efficiency: Fewer states can reduce the memory required to store transition information.
- Faster processing: A smaller transition structure can reduce processing overhead.
- Lower power consumption: Simpler hardware may require less energy.
- Reduced hardware complexity: Fewer state registers and simpler logic may be needed in a hardware implementation.



Conclusion: DFA minimization improves the efficiency of pattern validation by reducing unnecessary states while preserving the accepted language.

Question 2. Conversion of a Finite Automaton into a Regular Expression

Answer:

A regular expression represents a regular language using symbols and operators. A finite automaton can be converted into an equivalent regular expression so that both recognize the same set of strings.

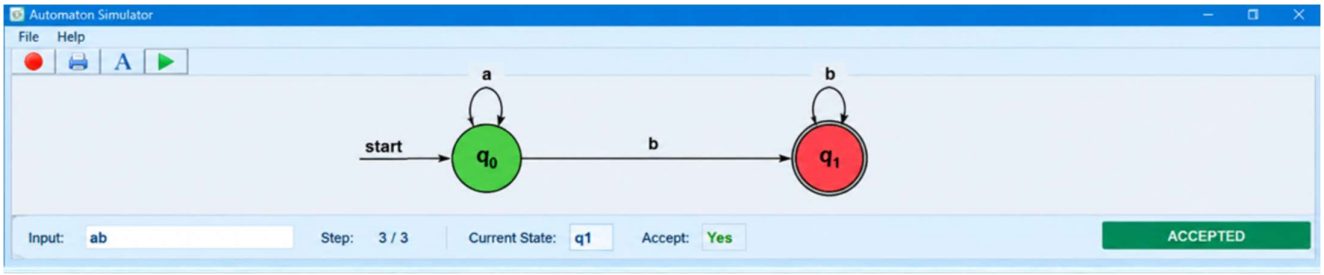
Steps for conversion

1. Identify the states: Find the initial state, final state or states, and all transitions of the given automaton.

2. Add a new start state: If necessary, connect a new start state to the original initial state using an (epsilon)-transition.
3. Add a new final state: Connect all original final states to a new final state using (epsilon)-transitions.
4. Eliminate intermediate states: Remove one state at a time and update the transition labels to preserve all possible paths.
5. Use regular expression operations: Union represents alternatives, concatenation represents sequences, and Kleene star represents zero or more repetitions.
6. Obtain the final expression: After eliminating all intermediate states, the label from the new start state to the new final state gives the equivalent regular expression.

Example

Consider a simple automaton with a loop on a at the initial state (q0), a transition on b from (q0) to the final state (q1), and a loop on b at (q1).



The equivalent regular expression is:

[boxed{a^*bb^*}]

This expression means zero or more a characters, followed by at least one b, and then zero or more additional b characters.

For example, b, ab, aab, and abb are accepted.

Conclusion: By eliminating states and updating transition expressions, a finite automaton can be converted into an equivalent regular expression.

Question 3: The given grammar is:

Answer:

[begin{aligned} S & rightarrow a Sc mid T mid U mid epsilon T & rightarrow b Tc mid epsilon U& rightarrow a Ub mid epsilon end{aligned}]

Here, (epsilon) represents the empty string.

A) What is the language defined by the grammar?

Answer:

The language is obtained by applying the production rules of the grammar.

- The production (S rightarrow a Sc) adds one a at the beginning and one c at the end.

- The production (T rightarrow b Tc) generates equal numbers of b and c.
- The production (U rightarrow a Ub) generates equal numbers of a and b.
- The production (S rightarrow epsilon), along with the empty productions of (T) and (U), allows the empty string.

Therefore, the language can be expressed as:

[boxed{ begin{aligned} L={}&{a^n b^m c^{m+n} mid n,m geq0} & cup {a^{n+k}b^k c^n mid n,k geq0} end{aligned}}]

The first set is generated through (T), and the second set is generated through (U). Both sets include the empty string when their parameters are zero.

Examples of strings generated by the grammar include:

[epsilon, ac, ab, aabb, aaabbc, bbcc]

B) Give the derivation tree for the word aaabbc

Answer:

The required word is:

[w=aaabbc]

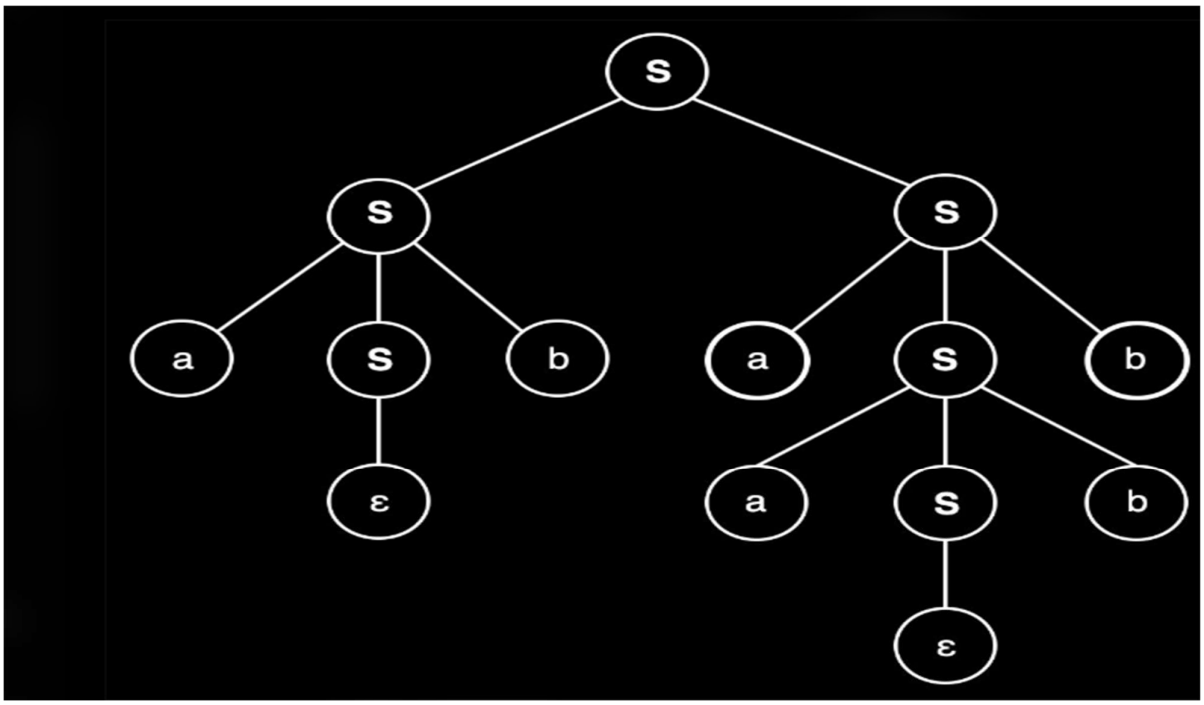
We can derive this word by using the productions (S rightarrow a Sc), (S rightarrow U), and (U rightarrow a Ub).

Leftmost derivation

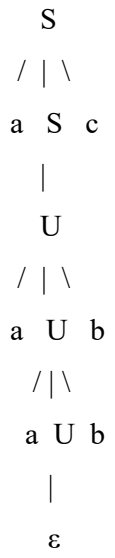
[begin{aligned} S& Rightarrow a Sc &Rightarrow a Uc &Rightarrow aaUbc &Rightarrow aa aUbbc &Rightarrow aaabbc end{aligned}]

In the last step, (U rightarrow epsilon) is applied. Hence, the generated word is aaabbc.

Derivation tree



The structure of the derivation tree can also be written as:



Reading the leaves from left to right and omitting (epsilon) gives aaabbc.

C) Is the grammar ambiguous? Justify your answer.

Answer:

A context-free grammar is ambiguous if at least one string in its language has two distinct parse trees or two distinct leftmost derivations.

The given grammar is ambiguous because the string ac can be derived in two different ways.

First derivation:

[S\Rightarrow aSc\Rightarrow a\epsilon c \Rightarrow ac]

Here, the inner (S\) is replaced directly by (\epsilon\).

Second derivation:

[S\Rightarrow aSc\Rightarrow aTc \Rightarrow a\epsilon c \Rightarrow ac]

Here, the inner \(S\) is replaced by \(T\), and then (T\) is replaced by (\epsilon\).

Both derivations generate the same string ac, but they use different production rules and produce distinct parse trees.

Therefore,

[\boxed{\text{The grammar is ambiguous.}}]

Conclusion: The grammar generates strings through different production paths, and the string ac has two distinct derivations. Hence, the grammar is ambiguous.

SIMATS ENGINEERING
ASSESSMENT TOOL

Question 4. Closure Property of Regular Languages and the Regularity of Prime-Length Strings

I) Show that if \(L_1\) and \(L_2\) are regular, then \(L_1\cup L_2\) is also regular.

Answer:

The union of two languages contains all strings that belong to either of the languages or to both of them.

Suppose (L_1\) and \(L_2\) are regular languages. Since they are regular, there exist finite automata (M_1\) and (M_2\) that recognize them.

We can construct a new NFA that recognizes their union by introducing a new initial state and connecting it to the initial states of (M_1\) and \(M_2\) using (\epsilon\)-transitions.

The new automaton can choose either automaton to process the input. If either automaton accepts the string, the new automaton accepts it.

Thus, the language recognized by the new automaton is:

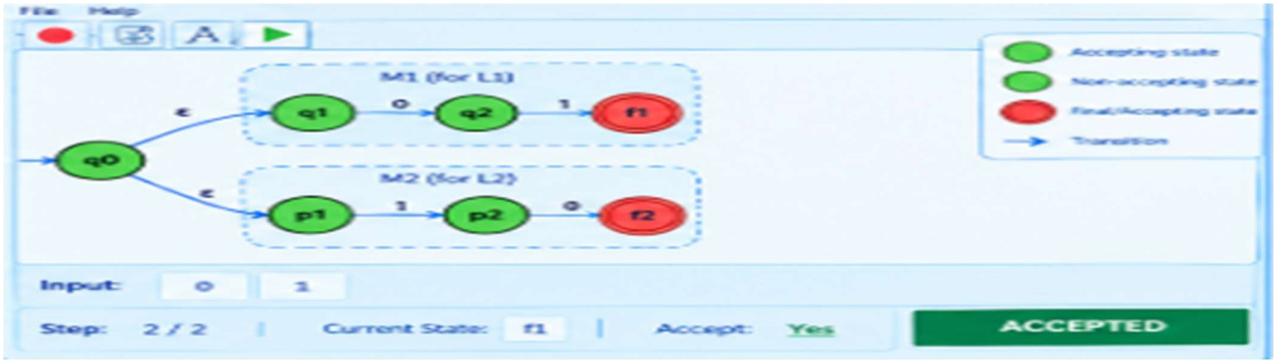
[L(M)=L_1\cup L_2]

Since an NFA recognizes a regular language, (L_1\cup L_2\) is also regular.

Therefore,

[\boxed{L_1,L_2\text{ regular}\Rightarrow L_1\cup L_2\text{ regular}}]

Conclusion: Regular languages are closed under union because a finite automaton can be constructed to recognize the union of any two regular languages.



II) Is the language \(\{a^p\mid p\text{ is prime}\}\) regular? Justify.

Answer:

The language is:

[L=\{a^p\mid p\text{ is prime}\} \]

It contains strings consisting only of a, where the length of the string is a prime number. Examples include aa, aaa, aaaaa, and aaaaaaa, with lengths 2, 3, 5, and 7 respectively.

We will prove that this language is not regular using the pumping lemma.

Step 1: Assume that the language is regular

Suppose \(L\) is regular. According to the pumping lemma, there exists a pumping length \(N\) such that any string \(s\) with length at least \(N\) can be divided into \(s=xyz\), satisfying:

[|xy|\leq N,\quad |y|>0]

and

[xy^iz\in L\quad\text{for every }i\geq0.]

Step 2: Choose a suitable string

Choose a prime number (q>N). Consider the string:

[s=a^q]

Since (q) is prime, (s) belongs to (L).

Step 3: Apply the pumping lemma

Because (|xy|\leq N\) and the string contains only a, the substring (y) consists of (k) occurrences of a, where (k\geq1\).

Now choose (i=q+1\). The length of the pumped string becomes:

[\begin{aligned} |xy^{q+1}z| &=q+qk\\ &=q(1+k) \end{aligned} \]

Since \(q\geq2\) and \(k\geq1\), both factors are greater than 1. Therefore, \(q(1+k)\) is composite, not prime.

Hence,

[xy^{q+1}z\notin L]

This contradicts the pumping lemma, which requires the pumped string to remain in \(L\).

Conclusion

Our assumption that the language is regular leads to a contradiction. Therefore,

[boxed{{a^p\mid p\text{ is prime}\}\text{ is not regular}}

The language is not regular because a finite automaton cannot recognize exactly those unary strings whose lengths are prime.